

Investigating the Effects of File System Architecture on Attacker Behavior

The Barbs

Group 2G

HACS200

Aaryan Patel, Abi Guicheteau, Eileen Chen, Maria House Ordonez, Parthav Poudel

1. EXECUTIVE SUMMARY

Over the course of a semester, the group sought to answer the following question: how does file system architecture affect attacker behavior? More specifically, the experiment focused on the type and number of commands executed by attackers. The null hypothesis was that the file system architecture would have no effect on attacker behavior, meaning that the file design layout would not cause significant differences in commands run by attackers. There were three different file designs: direct, linear, and tree, which are further explained in Section 3 of this paper. Two alternative hypotheses were also developed. The first was that tree architecture would receive the highest level of interaction and log the greatest number of commands, and the second alternative hypothesis was that the linear architecture would receive fewer commands of the malware category since a larger variety of commands is required to navigate through the system of files in the honeypot.

A large quantity of data was collected throughout the experiment, primarily focusing on commands run by attackers. The total number of commands across all honeypots and architectures was collected along with the total number of commands for each architecture. For each file structure, the number of commands for each command category were also recorded. These categories were data extraction, navigation, system, privilege escalation, and network/malware. If a command did not fall into one of those categories, it was placed into a trash category.

With the data collected, the group failed to reject the null hypothesis and concluded that file architecture has no impact on the commands executed by attackers. There was no statistically significant evidence to support either of the alternative hypotheses.

2. BACKGROUND RESEARCH

A honeypot is a closely monitored network decoy that can distract adversaries from valuable machines on the network, give an early warning about exploitation trends, and detailed information about adversarial behavior during honeypot exploitation. The honeypots focus on the last role, allowing observation of attacker behavior as they interact with the carefully configured systems. Previous studies utilizing honeypots often involve deliberate file system configuration and honey placement, insinuating that file organization plays a major role in honeypot interaction. A honeypot configured to analyze a type of Wireless Personal Area Networks called Zigbee intentionally held Wireshark pcap files stored at random deposits in the file system. The intended purpose was maintaining the front that these files were misplaced by an uninformed administrator, demonstrating that attackers can draw observations about the file system and alter their behavior (Dowling, 2017). This study led to the reasoning that attackers would similarly behave differently when confronted with different file system structures.

A study on the role of system properties on attackers' behavior observed how the file population on a compromised system had an effect on human and automated attacker behavior. It was concluded that a higher file population results in differences in behavior between humans and automated attackers (Barron, 2017). This suggests that significant differences exist from human to human when interacting with different file systems. The importance of choosing an optimal file organization is emphasized by the associated costs of programming or reprogramming (Cardenas, 1973). This choice can influence storage costs and the average time required to retrieve information. The study conducted by the University of California, Los Angeles yields statistically significant differences in the performance of basic file organizations supported by present day computing systems: the well-known sequential, random, and indexed

sequential organizations. Additionally, a study conducted at the International Conference on Cyber Conflict discusses how commands can be categorized as inherently malicious or not, and how this classification can be modeled by a machine learning program (Dumont, 2019). This shows that different classifications of commands exist, some malicious and some not.

Consolidating the information from the sources led to the composition of the research question. The experiment aims to explore the possibility of differing file architectures impacting the categories of commands executed by attackers, and how it could possibly hinder their activity in a compromised computer system. Based on research of previous experiments, this topic has yet to be explored in depth.

3. RESEARCH QUESTION AND HYPOTHESIS

The research question created at the beginning of the semester was investigating how file system architecture affects attacker behavior within the honeypots. It was hypothesized that file architectures of greater complexity would cause significant differences in attacker behavior. The null hypothesis was that the number of commands in each category and the overall level of interaction would not differ between file systems, with the reasoning that file architectures do not actually affect a hacker's commands. The alternative hypotheses were: (1) the tree architecture would receive the highest level of interaction and log the greatest number of commands, and (2) the linear architecture would receive fewer commands of the malware category since more effort was required to navigate through the system.

4. EXPERIMENT DESIGN

4.1 Container Setup

In the experiment, Linux LXC containers were utilized as they would allow quick set up and take down of the honeypots as necessary. As containers run atop the host operating system and receive their own file systems, they enforce containment of malicious activity; damage can be quickly mitigated by removing the container and simply setting it up again. To monitor activity within the honeypot, we utilized the ACES Man-In-The-Middle Server, which records every key entered by the attacker. The service creates a log file in the host VM, allowing constant observation of the honeypots. Forever was also installed in the host VM, allowing the MITM server to run properly on a forever process.

To evaluate the hypothesis outlined in Section 4, honeypots were created to contain seemingly valuable data. Every honeypot was consequently modeled to contain a file system replicating one that would be on a machine used in a bank. Each honeypot contained large command separated values (CSV) files of fake, randomly generated employee information regarding names, emails, usernames, passwords, addresses, transactions, and account balances. To generate the fake bank information, the Faker Python library was utilized which allows options for fabricating basic personal information. The information was not entirely randomized, meaning that addresses were solely from the United States and there were few diverse names. However, this was used to bolster the credibility of the fake bank as it underlined the idea that the organization served customers from a specific area and culture. After generating the information, a base copy was stored in the host machine. As new containers were created, the same files were uploaded to their respective locations within the honeypots. This control eliminated any possibility that different file contents would alter attacker behavior between honeypots.

For the experiment, five public IP addresses for the University of Maryland's public network were received. This facilitated the deployment of five high-interaction honeypots simultaneously. The experimental design utilized three different honeypot configurations, with each configuration organizing the honey in the following file architectures: direct, sequential, and tree. The first configuration contained a Direct File Architecture (*Figure 1*), defined as having no organization of data. Each CSV file was placed into a separate directory, all of which exists independently in a singular, overarching directory. In doing so, attackers had equal and direct access to each individual CSV file. The second configuration contained a Sequential File Architecture (*Figure 2*), defined as having files and data organized in a linked list of directories. Attackers will enter from one directory, and must traverse directories in a linear, sequential fashion to reach the end. There was only one direction to proceed to access the honey. The third and final configuration contained a Tree File Architecture (*Figure 3*), defined as having files and data organized in a similar fashion to a tree data structure in common programming languages. Attackers begin in the root directory and are required to traverse through subtrees of directories to locate CSV files.

4.2 Recycling Process

To create the containers themselves, a base container was created with all of the required packages for the honeypots. Open-ssh was also installed. When creating new containers, the contents of the default container was simply copied into the new container. This copying process was substantially less time-consuming than fully installing every service every time a container was created. To manage the containers themselves, a recycling script was created in the host VM. The script received the IP address of the container to be created as a parameter, as well as a

random ephemeral port number to run the MITM Server on. The script checked that the container with the specified IP address had not already been created and was running- if it did, the script deleted this container before moving on.

Within an infinite loop, the containers and appropriate firewall and routing rules were configured. The script shuffled the three possible architectures – direct, sequential, and tree – and randomly assigned an architecture to the container. Separate scripts to perform various functions, such as loading the honey into the appropriate locations and creating log files for data collection (further explained in Section 6), were incorporated in the main recycling script to partition functionality. The MITM server incorporated auto-access, meaning that attackers would be able to set their credentials to log into the containers. Finally, the last step in honeypot setup required the establishment of prerouting and postrouting Network Address Translation rules between the honeypot and host VM, as well as a rule to redirect incoming SSH traffic towards the MITM Server.

The script maintained a conditional flag for whether the honeypot had been running for a predetermined period of time. While the honeypot was running, another conditional flag recorded whether an attacker had connected with the honeypot by searching for a specific line in the MITM log file. If an attacker had connected, then the script inserted the appropriate iptable rules to ensure that another attacker could not connect to the honeypot while the original attacker was still connected. Immediately after the original attacker disconnected, these new rules were deleted to allow connections from other attackers.

The total time since the beginning of an attacker session was constantly calculated and updated, and the recycling process was initiated once that time exceeded thirty seconds.

However, in the event a honeypot received no attacker connections, the recycling script implemented a hard cut-off at thirty minutes. Utilizing an infinite loop within the recycling script allowed the deployment of the honeypots to be an autonomous process: the recycling script was run once per public IP address upon reboot using cron and required no further intervention.

As specified in Section 3, the time the containers ran was altered to increase the amount of data collected. After the conditions had been met for the containers to be recycled, the recycling script removed the forever processes running the MITM server and destroyed the container. Network Address Translation rules were deleted in order to prepare for the subsequent honeypot deployment.

The method in which the data was stored will be covered later in Section 6, as it was heavily intertwined with the data collection process. All scripts created to aid in the process of data collection and analysis were stored on the Host VM. For data backups, the data was periodically downloaded onto personal machines before substantial changes to the recycling or data collection process.

5. EXPERIMENT DESIGN CHANGES

Netdata was used to monitor all honeypots, and the application was installed onto the host VM. All honeypots contained the exact same data or honey, which was created with a Python script that generates bank information such as random fake names, and email addresses. The honeypots emulated a bank's digital filing system, containing information about employees and customers. The only factor that differs between the honeypots is the file architecture.

The honeypots were initially set to recycle in any of the following instances: (1) the attacker leaves the honeypot, (2) the attacker has been active in the honeypot for 24 hours, or (3)

the attacker has been idle in the honeypot for 12 hours. However, this was changed on October 22nd so that the honeypots would recycle after 10 minutes, and any attacker in the honeypot would be removed. This adjustment was made after the observation that shorter recycling times would result in more data. Dr. Cukier also recommended this implementation to the entire class. On October 24th, the recycling time was further shortened to three minutes for the same reasons.

On November 5th, the recycling script was changed again to recycle honeypots 30 seconds after attackers logged into the container or after the honeypot received 30 minutes of zero interaction. Finally, the last alteration to the recycling script implemented the creation of a base container with open-ssh installed, and then creating a copy of that container in the recycling script.

For data collection, the data collection script was initially written to utilize log files to determine if an attacker's command was one of five categories: data extraction, navigation, system, privilege escalation, or network/malware. The command and its corresponding category was placed into a dictionary, and if a future command was not recognized, it was categorized into the "other" text file. During the week of November 5th, data collection was changed to a new format in which folders were created for each day and contained the MITM logs for the five minute intervals. The data collection script was changed on November 26th to handle cases where attackers ran multiple commands in one line separated by semicolons, and in class on November 30th data collection was stopped and the honeypots were undeployed.

On the same day, a script was used to iterate through all MITM logs to manually categorize commands by typing in a number corresponding to one of the five categories or the trash category. Due to the setup of the command collection process, repeated commands were only required to be accounted for once. We consequently categorized a total of 320 commands.

6. DATA COLLECTION

For each attacker event, an MITM server was used to generate a single log file to collect ssh related information, including the attacker's IP address and the credentials they used to log into the container. Any keystrokes and commands run interactively or non-interactively were collected as well with the respective timestamps. Log files were stored in folders organized by architecture type, then day. During data collection from 10/22/23 to 11/30/23, 47,055 log files were gathered with a total number of commands of 69,605.

To analyze the type of commands attackers used, attacker commands were split into five different categories: system, network manipulation/malware installation, data extraction, navigation, and privilege escalation or persistence. These commands were created based on previous research investigating types of attackers in honeypots (Barron, 2017). Brief summaries and examples of each category are provided below in *Figure 6*. Command categorization was updated whenever an attacker entered a new command not already in the pre-defined dictionary. Every command was categorized by at least two people to mitigate potential biases.

While the honeypot was active, the primary idea behind data processing was to (1) extract attacker commands, (2) compare the commands with a dictionary, (3) categorize the commands that matched and list the counts into an event file, and (4) add the commands that didn't match to an "uncategorized" database. Uncategorized commands were manually categorized and added to the dictionary, then removed from the database every day. At the end of the day, a script was run to recategorize all event files to reflect the updated dictionary. The specific implementation of this process varied throughout data collection. Event files generated during this time were primarily

used for monitoring, gathering preliminary findings, and modifying the generation or collecting scripts should significant issues occur. In order to ensure all event files were consistent and processing was the same for every log file, data extraction specifically used for analysis occurred after the honeypot was undeployed.

For every attacker event, the associated log file was parsed to extract each command. Commands executed consecutively within a single line were also parsed and separated by the presence of a semi-colon, piping, or “&&” delimiter. This was done to gather more data and to avoid confusion during categorization. The entirety of the command was used for categorization, including flags and any arguments. Originally, commands were automatically parsed and the first field was extracted to represent the category, but this created complications in the presence of header commands (sudo, /bin/bash).

In order to ensure every command could be categorized, a script was run after data collection stopped to output a list of every unique command entered, which totaled to be 320 commands, 11 of which were trashed. These unique commands were then manually categorized into a new dictionary file. Categorization took approximately an hour to complete. Event files were all regenerated using the new dictionary. CSV files were generated for every command category (five files in total) with three columns representing the different file architectures. An entry was defined as the count of commands under that specific category for a single log file. Zeros were included within the final data set.

No datapoints were eliminated after the initial honeypot deployment. Despite there being small adjustments in the recycling and post processing data collection scripts, they did not impact the research question and attacker behavior did not change significantly enough to justify the discarding of data.

7. DATA ANALYSIS

After extracting the filtered data from the raw dataset, Python libraries like Scipy and Plotly were utilized due to their efficiency in handling large datasets and robust statistical capabilities. Scipy provided reliable tools for conducting complex statistical tests such as the Kruskal-Wallis, ensuring precision in the analysis. Plotly visualized data with interactive and detailed charts, aiding in clearer interpretation of the results.

During data analysis, outliers in the data were identified for the number of commands executed by an attacker during their session. The presence of these outliers could have skewed results, and outliers could be a result of particularly aggressive attackers or automated attacker scripts with a large number of commands to run (many of which are in one line separated by semicolons or pipes). Given the non-normal distribution of the data, five Kruskal-Wallis tests were conducted, with one test per command category, between the three file architectures. This test aligned with the data as it does not assume a normal distribution and was less sensitive to outliers compared to tests like ANOVA. The p-value for each test assisted in determining statistically significant differences in the number of commands executed within each command category between file architectures. Comparing the p-value against a standard significance level, 0.05, allows for a conclusion on whether the differences in command execution among the three architectures are statistically significant or not for each command category. A p-value less than 0.05 would indicate that at least one of the file architectures leads to a significantly different number of commands being executed by attackers.

After conducting the 5 Kruskal-Wallis Tests, the p-values for each command category are as follows: 0.603 for data extraction, 0.611 for system information, 0.500 for privilege escalation,

0.288 for network/malware, and 0.435 for navigation (*Figure 4*). The p-values for all 5 command categories were above the 0.05 threshold, meaning that there were no significant differences between the mean for command categories among the three different file architectures. Therefore, the null hypothesis failed to be rejected. However, the data collected still offered insights that could be further explored. Notably, the most frequently executed command (31,455 times) across all file architectures was “uname -s -v -n -r -m”, a command typically used to gather system information. This pointed to a primary interest of attackers in understanding the system environment they have infiltrated.

Figure 5 represents how System Information commands were executed the most (32,797 times), followed by privilege escalation (23,511 times). This suggested that attackers prioritize gaining an understanding of the system and escalating their privileges within it. Conversely, commands related to network/malware (906 times) and data extraction (551 times) were executed less frequently, possibly indicating that these actions are secondary objectives or dependent on the successful execution of system information and privilege escalation commands. Navigation commands were executed 11,680 times, underscoring their role as a fundamental part of attackers' exploratory behavior within different file architectures. This observation could imply that regardless of the file structure, attackers invest effort in navigating through the system.

With 309 unique commands identified in the dataset, the diversity of the command pool used by attackers was noteworthy. It reflected the varied tactics and techniques employed by different attackers or by the same attackers under different circumstances. This diversity in commands could be further analyzed to understand if certain types of file architectures encourage a broader array of attack strategies. Given these insights, several alternative questions arise for potential future research. Is there a relationship between the complexity of the commands used

by attackers and their frequency? Do attackers tend to use more complex commands less frequently, and if so, what might this indicate about their skills or objectives? Is there any correlation between the attackers' origin (if identifiable given the nature of many attackers using IP masking techniques like virtual private networks) and their behavior or the types of commands used? The diversity in command usage among attackers highlights complexity where attackers' choices and strategies were not strictly influenced by system architecture. This leads to deeper questions about the nature of these attacks—questions that explore the complexity of commands and the potential link between attackers' origins and their methods.

8. CONCLUSION

The core of this study was centered around the hypothesis that file architecture of varying types would influence attacker behavior. This was influenced by prior studies that file organization is a critical factor in honeypot interaction and that attackers modify their strategies based on the structure they encounter. However, the outcomes of the experiment showed a different result. The statistical analysis, specifically Kruskal-Wallis Tests, revealed that the differences in command execution across file architectures were not significant. File system architecture might not be as pivotal.

If presented with the opportunity to continue the study, additional types of file architectures beyond the direct, sequential, and tree structures would be explored to provide a broader perspective on file organization and attacker behavior. This could involve experimenting with more complex or less common file structure to see if they elicit different responses from attackers. Employing machine learning to analyze the data collected, rather than a manual analysis with bash scripting, can automate many processes and uncover patterns and correlations

that are previously missed. It could be particularly useful in classifying and predicting attacker behaviors based on different system variables. Furthermore, investigating the potential correlation between the geographic or network origin of the attackers (as much as can be determined) and their chosen strategies could provide insights into whether attack patterns are regionally specific or globally consistent.

While the final result challenged the initial hypothesis, it lays groundwork for future research on cybersecurity vulnerabilities based on system characteristics. They also highlight the importance of considering a broader range of features beyond file architecture such as system configurations or network setups as shown by the most frequent commands executed by attackers being ones that output system information. By gathering information on the system, attackers are able to gauge its vulnerability. Therefore, with the findings, much is to be studied with varying honeypot system characteristics.

APPENDIX A

Many lessons were learned in the process of conducting this experiment. Firstly, it was necessary to learn how to construct adaptable and scalable code. Many issues arose with the honeypots post-deployment. For example, a few weeks into data collection an issue arose where the forever processes running the MITM Servers would enter a stopped state, and duplicate processes would be made. The group worked to find the issue and incorporate a solution into the already existing recycling script and make it adaptable. Additionally, writing scalable was highly important as it reduced debugging time. A recycling script for a singular honeypot was used, allowing for a large number of honeypots to be easily created (given the required number of public IP addresses). Secondly, scripts were also reconfigured to increase efficiency. Previously,

the scripts took on a brute force approach that were not very efficient, but produced the desired solution. However, as the data collected accumulated massively throughout the semester, these scripts took a large amount of time to run. The group had to discover more efficient ways to accomplish the same task.

This experiment provided valuable real world experience, allowing the group to strengthen scripting skills and increasing confidence with Bash. Additionally, processing the data provided valuable experience with data analytics and hypothesis testing. However, for people in the class with no statistical experience at all, it presented a major obstacle to their groups.

The group was surprised to discover that most interaction with the honeypots originated from bots, especially considering how the hypothesis was dependent upon human interaction with the honey. Additionally, there were a large number of commands that were unfamiliar to the group. For some commands, it was very difficult to find official documentation regarding their purpose. Finally, the group was surprised to discover that attacker session times were considerably short, and all actions taken by attackers occurred within a thirty second time frame.

The experiment did not encounter major pitfalls or failures that severely impacted the data collected. However, the group ran into minor bugs and issues that arose during the data collection period. Thus, solutions had to be implemented during the height of data collection, which slightly impacted the integrity of data previously collected before the issue arose. In spite of these issues, the group was able to steadily collect data throughout the entirety of the data collection period.

APPENDIX B

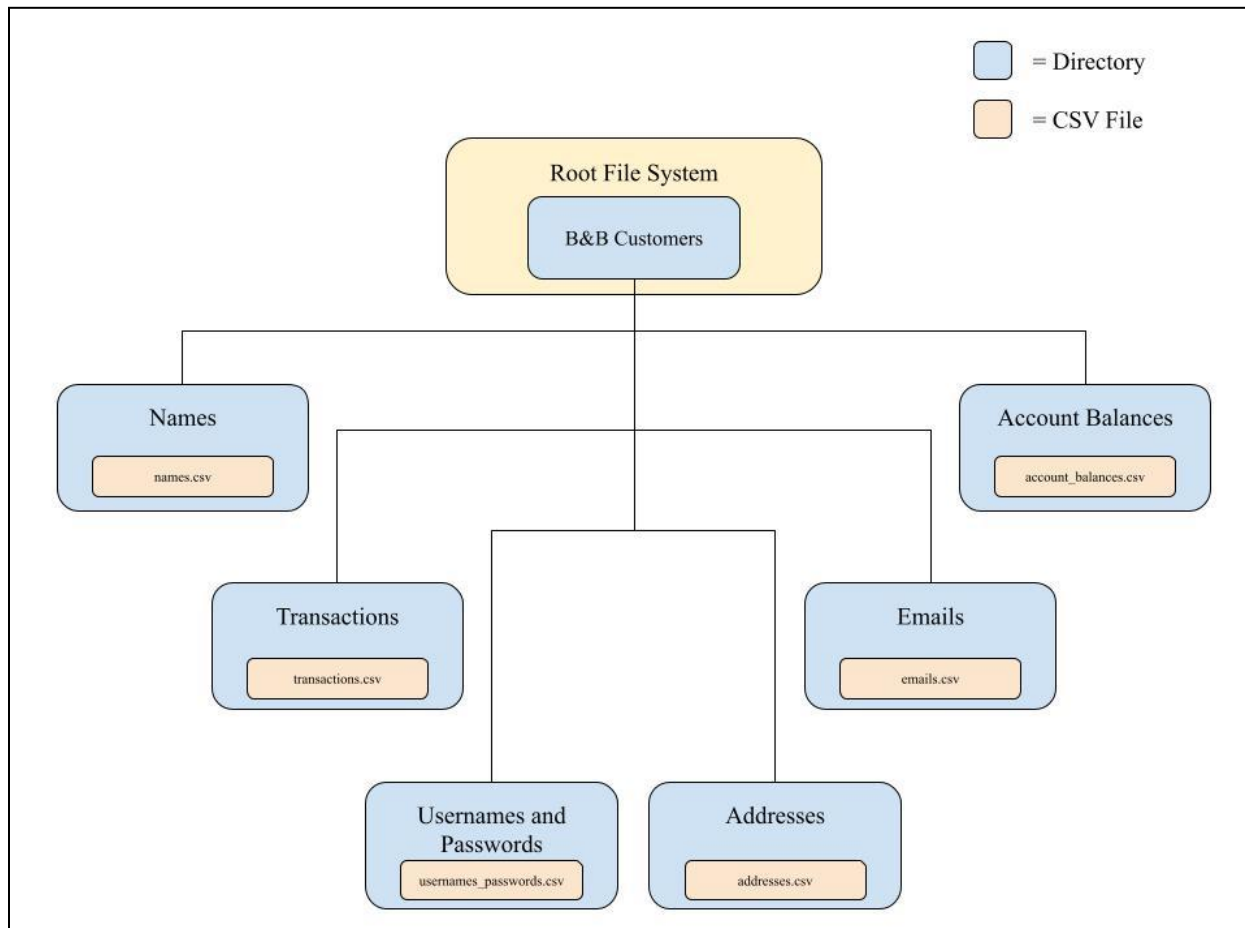
**Figure 1.** *Direct Architecture Layout*



Figure 2. *Sequential Architecture Layout*

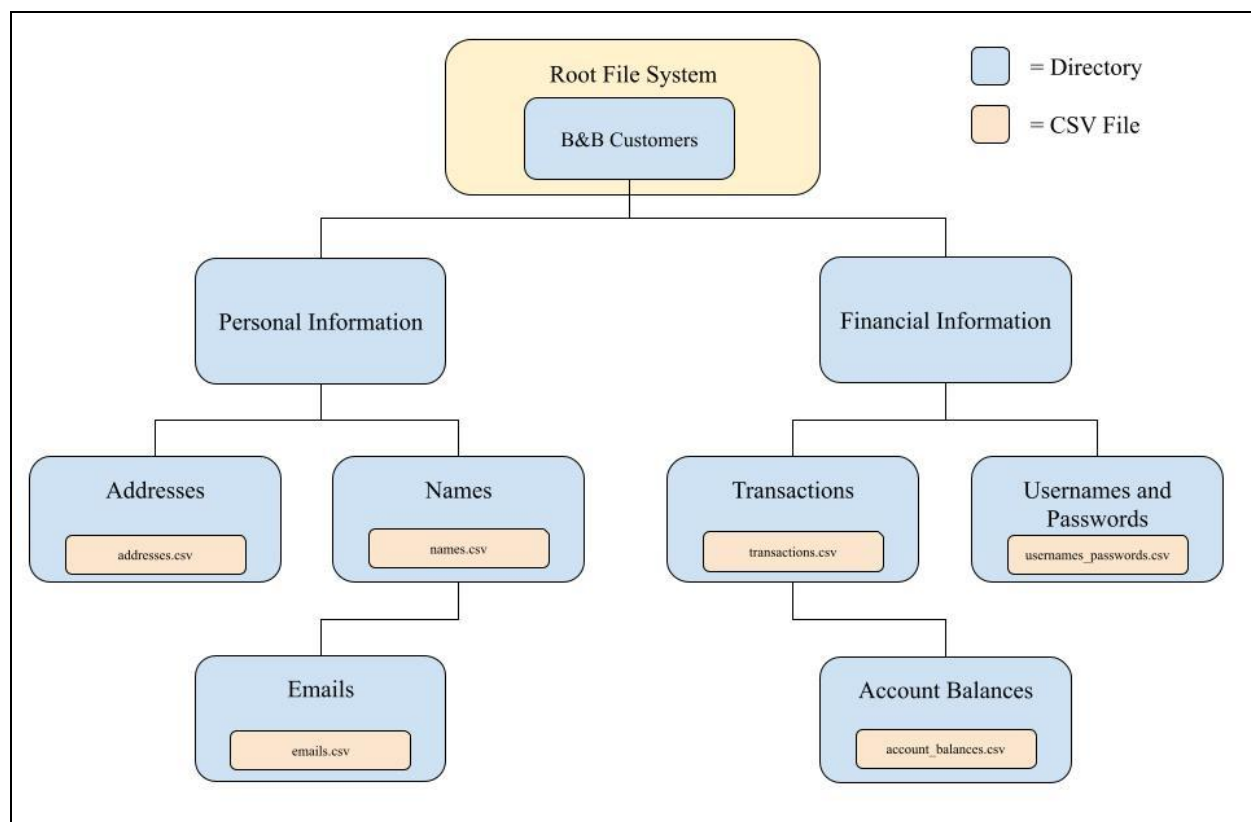


Figure 3. *Tree Architecture Layout*

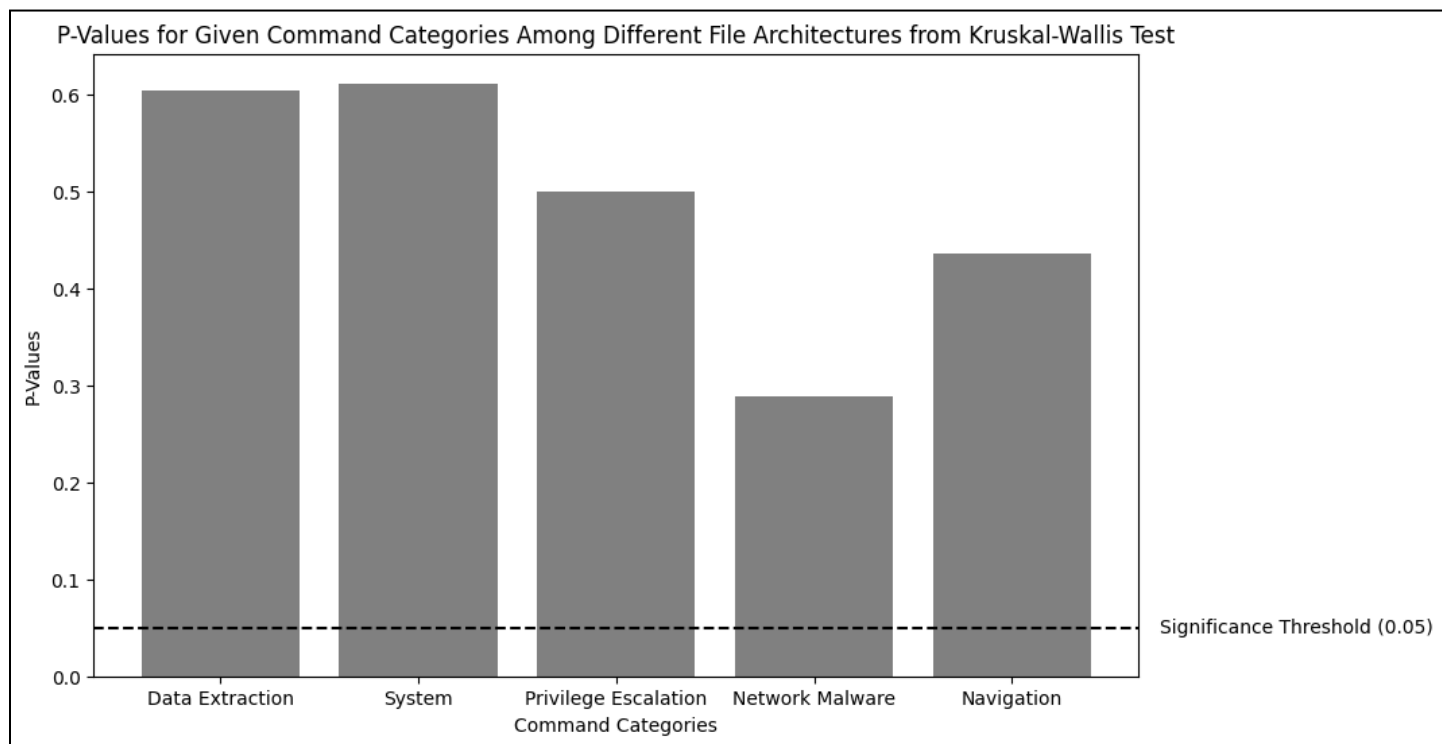


Figure 4. *Results of Kruskal-Wallis Tests*

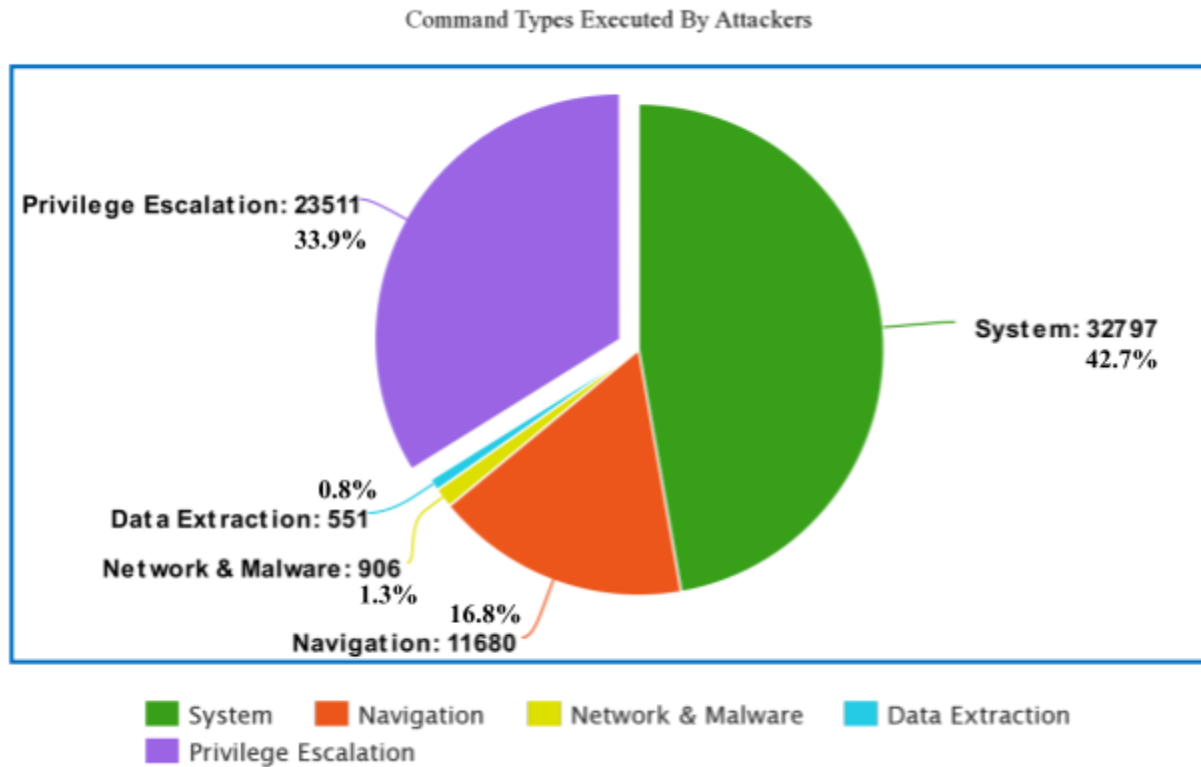


Figure 5. *Pie Chart Comparing Number of Commands Executed by Category*

Category Type	Description	Example Commands
System Information	Commands that gather information about the system	<i>id, ps aux, uname</i>
Network Manipulation/ Malware Installation	Downloading or executing malicious software on system	<i>wget, curl</i>
Data Extraction	Compress or transfer data from or to the system	<i>zip, scp</i>

Navigation	Commands used to navigate the file system, list directory contents, or search for files	<i>cd, ls, grep, pwd</i>
Privilege Escalation	Commands used to escalate privileges - often gaining root access and modifying system configurations	<i>sudo su, crontab, chmod, systemctl</i>

Figure 6. *Table of Command Categories*

APPENDIX C

recycle.sh: Main recycling script:

```
#!/bin/bash

if [ $# -ne 2 ]
then
    echo "Usage: [IP Number] [Random Port Number]"
    exit 1
fi

if [[ ! $1 =~ ^[0-9]+\.[0-9]+\.[0-9]+\.[0-9]+$ ]]; then
    echo "Error: invalid IP"
    exit 2
fi

IP=$1
port_num=$2

lastgroup=$(echo $IP | cut -d'.' -f4)
container_name="BB_Server_"$lastgroup
exists=$(sudo lxc-ls $container_name | grep $container_name -c)

# Stops and destroys container if it already exists
if [ "$exists" -eq 1 ]
then
    sudo lxc-stop -n $container_name
    sudo lxc-destroy -n $container_name
fi

# Runs once to install forever into system and activate eth1 interface
sudo ip link set dev "eth1" up

while true
do
    # randomly selects config folder from data directory
    con=$( ls /home/student/data | shuf -n 1 )

    # Copy base container with openssh installed
    sudo lxc-stop -n base
    sudo lxc-copy -n base -N $container_name
    sleep 10
    sudo lxc-start -n $container_name
    sleep 10

    # transfer honey to container
    if [ $con == "dir" ]
    then
        /home/student/create_direct.sh $container_name
    elif [ $con == "seq" ]
    then
        /home/student/create_linear.sh $container_name
    else
        /home/student/create_binary.sh $container_name
    fi
fi
```

```

sleep 5

day="Day_$(date +%m_%d_%y)"
if [ ! -d /home/student/data/$con/$day ]
then
    sudo mkdir /home/student/data/$con/$day
fi

containerIP=$(sudo lxc-info -n $container_name -iH)
logfile=data/$con/$day/$(date +%m_%d_%y_%H_%M_%S)

# start MITM server
sudo /usr/bin/forever --id $container_name --pidFile
/home/student/pids/$container_name.pid -l /home/student/"$logfile" start
/home/student/MITM/mitm.js -n $container_name -i $containerIP -p $port_num
--auto-access --auto-access-fixed 1 --debug

sleep 5

# PREROUTING & POSTROUTING rules
sudo sysctl -w net.ipv4.conf.all.route_localnet=1
sudo ip addr add $IP/24 brd + dev eth1
sudo iptables --table nat --insert PREROUTING --source 0.0.0.0/0
--destination $IP --jump DNAT --to-destination $containerIP
sudo iptables --table nat --insert POSTROUTING --source $containerIP
--destination 0.0.0.0/0 --jump SNAT --to-source $IP

# additional MITM redirection
sudo iptables --table nat --insert PREROUTING --source 0.0.0.0/0
--destination $IP --protocol tcp --destination-port 22 --jump DNAT
--to-destination "127.0.0.1:$port_num"

flag=true # flag to signify recycling
attacker_flag=false # flag to determine if attacker is present in honeypot
last_line=1 # represents the line in the MITM log where the last attacker
entered
start=$(date +%s) # changes after first attacker enters
initial_time=$(date +%s) # does not change

first_attacker=false # flag to determine if FIRST attacker has entered
attacker_ip="NONE"

monfile="/home/student/monitoring/"$container_name
# separate monitoring file to see variable states

# while container is within time frame
while $flag
do
    cat /dev/null > $monfile # clear monitoring file
    echo $'\n'"Temp values before end of while: " >> $monfile

    curr=$(date +%s) # current time

    if sed -n "${last_line},\${p}" /home/student/"$logfile" | grep -q "Attacker
authenticated and is inside container"
    then

```

```

if [ "$attacker_flag" != "true" ]
then
    if ! $first_attacker
    then
        first_attacker=true
        start=$(date +%s) # Time of first attacker entry.

        # outputs the time stamp for each attacker entry
        ip_file="/home/student/ip_data/ip_$(echo $IP | cut -d"." -f4).txt"
        echo "$(date +%D %T)" $start $logfile" >> $ip_file
    fi

    attacker_ip=$(sed -n "${last_line},\${p}" /home/student/"$logfile" |
grep "Attacker connected" | tail -1 | cut -d' ' -f8)

    # Ensuring that two attackers can't enter honeypot simultaneously
    sudo iptables --insert INPUT --protocol tcp --source 0.0.0.0/0
--destination "127.0.0.1" --destination-port $port_num --jump DROP
    sudo iptables --insert INPUT --protocol tcp --source $attacker_ip
--destination "127.0.0.1" --destination-port $port_num --jump ACCEPT
    attacker_flag=true
fi
fi

# delete rules for only allowing same attacker into honeypot once
connection is closed
if $attacker_flag
then
    if sed -n "${last_line},\${p}" /home/student/"$logfile" | grep -q
"Attacker closed connection"
    then
        attacker_ip=$(sed -n "${last_line},\${p}" /home/student/"$logfile" |
grep "Attacker connected" | tail -1 | cut -d' ' -f8)

        sudo iptables --delete INPUT --protocol tcp --source 0.0.0.0/0
--destination "127.0.0.1" --destination-port $port_num --jump DROP
        sudo iptables --delete INPUT --protocol tcp --source $attacker_ip
--destination "127.0.0.1" --destination-port $port_num --jump ACCEPT

        attacker_flag=false
        last_line=$(cat /home/student/"$logfile" | grep -n -o
"$attacker_time" | cut -d":" -f1)
        last_line=$(( $last_line + 1 ))
    fi
fi

#Change time here as needed
enter_time=$(( $curr - $start ))
total_time=$(( $curr - $initial_time ))
total_time=$(( $total_time / 60 ))

# if time after log in exceeds 30 seconds than recycle
if [[ $enter_time -ge 30 && $first_attacker == "true" ]]
then
    flag=false
fi

# if there has been no interaction for more than 30 minutes, recycle

```



```

if [[ $total_time -ge 30 ]]
then
    flag=false
    curr=$(date)
    echo "$IP $(date +"%D %T.%N") $logfile" >>
/home/student/recycle_alerts.txt
fi

# OUTPUT variables to monitoring file
echo $container_name >> $monfile
echo $'\n'"Values at end of while: " >> $monfile
echo "Variables -----" >> $monfile
echo total time: $total_time >> $monfile
echo start: $start >> $monfile
echo last line: $last_line >> $monfile
echo attacker time: $attacker_time >> $monfile
echo attacker ip: $attacker_ip >> $monfile
echo "Flags -----" >> $monfile
echo first_attacker: $first_attacker >> $monfile
echo attacker_flag: $attacker_flag >> $monfile
sleep 5

done
# Destroying the honeypot, resetting NAT rules, and resetting MITM
sudo iptables --table nat --delete PREROUTING --source 0.0.0.0/0
--destination $IP --jump DNAT --to-destination $containerIP
sudo iptables --table nat --delete POSTROUTING --source $containerIP
--destination 0.0.0.0/0 --jump SNAT --to-source $IP
sudo iptables --table nat --delete PREROUTING --source 0.0.0.0/0
--destination $IP --protocol tcp --destination-port 22 --jump DNAT
--to-destination "127.0.0.1:$port_num"

sudo ip addr del $IP/24 brd + dev eth1

# sanity check for iptable
/home/student/check_iptable_once.sh $IP $port_num $logfile

sudo lxc-stop -n $container_name
sudo lxc-destroy -n $container_name

# Killing the forever processing running the MITM server by extracting the
unique ID we assigned to the MITM process
pid=$(cat /home/student/pids/$container_name.pid)
sudo rm -rf /home/student/pids/$container_name.pid
sudo /usr/bin/forever stop $pid
/home/student/categorize_commands.sh $logfile

done

```

get_unique_commands.sh: Gets unique commands which is then input into categorize_other.sh

```
#!/bin/bash

if [ $# -ne 1 ]
then
    echo "usage: ./get_unique_commands.sh [output file]"
    exit 1
fi

cat /dev/null > tempfile.txt
cat /dev/null > tempfile2.txt

for arch in data/*/
do
    cat <(grep "Noninteractive" $arch/* | cut -d' ' -f10-) <(grep "line from
reader" $arch/* | cut -d' ' -f9-) >> tempfile.txt
done

while IFS= read -r line
do
    IFS=";&|"
    read -ra array <<< "$line"
    for element in "${array[@]}"
    do
        echo "${element# *}" >> tempfile2.txt
    done
done < tempfile.txt

sort tempfile2.txt | sed 's/[[:space:]]*$// ' | uniq -c > $1

rm tempfile.txt
rm tempfile2.txt
```

kruskal_category: Script to generate the five CSV files used for data analysis:

```
#!/bin/bash

filepath="category_analysis/final/"

cat /dev/null > $filepath"dir.txt"
cat /dev/null > $filepath"tree.txt"
cat /dev/null > $filepath"seq.txt"

categories=("data" "nav" "sys" "priv" "net")

# creates a concatenated list of the number counts in every event file
for arch in data/*
do
    arch_str=$( echo $arch | cut -d '/' -f2 )
    for day in "$arch"/*
    do
        for log in "$day"/event_*
        do
            cat "$log" | sed -n '2,7 p' | rev | cut -d " " -f1 | rev >>
"$filepath""$arch_str"".txt"
        done
        echo "done $day"
    done
    echo "done $arch"
done

# gets every line mod count and concatenates the three architectures in
parallel with a comma delimiter (csv file)
count=1
for name in "${categories[@]}"
do
    paste <( cat $filepath"dir.txt" | awk -v var="$count" 'NR % 6 == var') <( cat
$filepath"seq.txt" | awk -v var="$count" 'NR % 6 == var') <( cat
$filepath"tree.txt" | awk -v var="$count" 'NR % 6 == var') >
"$filepath""$name"".txt"

    count=$(( count + 1 ))
done
```

categorize_other.sh: Script to categorize new commands with user input:

```
#!/bin/bash

if [ $# -ne 1 ]
then
    echo "Usage: /home/student/categorize_other.sh [file] "
    exit 1
fi

file=$1
categories=("system" "network_or_malware" "data_extraction" "navigation"
"privilege_escalation")

num_lines=$(cat $file | wc -l)
while [ $num_lines -gt 0 ]
do
    # display categories
    echo "1 = sys, 2 = net/mal, 3 = data, 4 = nav, 5 = priv, 6 = skip, 7 =
trash"

    # print first line of file
    sed -n 1p $file
    read -p "Enter category: " category

    if [ "$category" -ge 0 ] && [ "$category" -le 5 ]; then
        # append this to end of file of other txt
        echo "$(sed -n 1p $file) ${categories[category - 1]}" >>
/home/student/commands.txt
        sed -n '2,$'p $file > /home/student/temp.txt
        cp /home/student/temp.txt $file
        rm /home/student/temp.txt
        num_lines=$((num_lines - 1))
    fi
    # skip and move line to bottom of current file
    if [ "$category" -eq 6 ]; then
        sed -n '2,$'p $file > /home/student/temp.txt
        sed -n 1p $file >> /home/student/temp.txt
        cp /home/student/temp.txt $file
        rm /home/student/temp.txt
    fi

    if [ "$category" -eq 7 ]; then
        sed -i '1d' $file
        line=$(sed -n 1p $file)
        echo $line >> /home/student/trash.txt
    fi
done
```

categorize_commands.sh: Script to sort commands into categories:

```
#!/bin/bash

#creating dict
declare -A count_dict
count_dict["other"]=0
count_dict["data_extraction"]=0
count_dict["navigation"]=0
count_dict["system"]=0
count_dict["privilege_escalation"]=0
count_dict["network_or_malware"]=0

# This script will be called after a honeypot is recycled
if [ $# -ne 1 ]
then
    sudo echo "Usage: logfile"
    exit 1
Fi

file_architecture=$(sudo echo $1 | cut -d"/" -f5)
date=$(sudo echo $1 | cut -d"/" -f7)
sudo touch /tmp/processing$date
sudo chmod +w /tmp/processing$date

grep "line from reader" $1 | cut -d" " -f9- | sudo tee -a /tmp/processing$date
> /dev/null 2>&1

grep "Noninteractive mode attacker command" $1 | cut -d" " -f10- | sudo tee -a
/tmp/processing$date > /dev/null 2>&1

count=0
commandFile="/home/student/just_commands.txt"
awk 'BEGIN{FS=OFS=" "}{NF--; print}' /home/student/commands.txt > $commandFile
cat /dev/null | sudo tee -a /home/student/other_check.txt > /dev/null 2>&1

while IFS= read -r line
do
    IFS=";&|"
    read -ra array <<< "$line"
    for element in "${array[@]}"
    do
        count=$(( $count + 1 ))
        command=$(echo "${element# *}" | sed 's/[[:space:]]*$/')
        if ! grep -qFx "$command" /home/student/trash.txt
        then
            if [ ! -z $command ]
            then
                line_num=$( cat $commandFile | grep -Fnx "$command" | head -1 | cut
-d'::' -f1)
                if [ ! -z $line_num ]
                then
                    category=$(sed -n "${line_num}p" /home/student/commands.txt | rev
| cut -d" " -f1 | rev)
                    category=${category%"${category##*}[![:space:]]"}

```

```

        # increment counter value in the dictionary if command already
found
        count_dict["$category"]=$(( count_dict["$category"] + 1 ))
    else
        if ! grep -q "$command" /home/student/other.txt
        then
            sudo echo "$command" | sudo tee -a /home/student/other.txt >
/dev/null 2>&1
        fi
        count_dict["other"]=$(( count_dict["other"] + 1 ))
        sudo echo "$command $1" | sudo tee -a
/home/student/other_check.txt > /dev/null 2>&1
    fi
fi
done
done < /tmp/processing$date

day=$(sudo echo $1 | cut -d"/" -f6)

if [ ! -d /home/student/data/"$file_architecture"/"$day" ]
then
    sudo mkdir /home/student/data/"$file_architecture"/"$day"
Fi

if [ -f /home/student/data/"$file_architecture"/"$day"/event_$date ]
then
    sudo rm /home/student/data/"$file_architecture"/"$day"/event_$date
fi

echo "Categories:" | sudo tee -a
/home/student/data/"$file_architecture"/"$day"/event_"$date" > /dev/null 2>&1
echo "Data Extraction: ${count_dict["data_extraction"]}" | sudo tee -a
/home/student/data/"$file_architecture"/"$day"/event_"$date" > /dev/null 2>&1
echo "Navigation: ${count_dict["navigation"]}" | sudo tee -a
/home/student/data/"$file_architecture"/"$day"/event_"$date" > /dev/null 2>&1
echo "System: ${count_dict["system"]}" | sudo tee -a
/home/student/data/"$file_architecture"/"$day"/event_"$date" > /dev/null 2>&1
echo "Privilege Escalation: ${count_dict["privilege_escalation"]}" | sudo tee
-a /home/student/data/"$file_architecture"/"$day"/event_"$date" > /dev/null
2>&1
echo "Network or Malware: ${count_dict["network_or_malware"]}" | sudo tee -a
/home/student/data/"$file_architecture"/"$day"/event_"$date" > /dev/null 2>&1
echo "Other: ${count_dict["other"]}" | sudo tee -a
/home/student/data/"$file_architecture"/"$day"/event_"$date" > /dev/null 2>&1
echo " ----- " | sudo tee -a
/home/student/data/"$file_architecture"/"$day"/event_"$date" > /dev/null 2>&1
cat /tmp/processing$date | sudo tee -a
/home/student/data/"$file_architecture"/"$day"/event_"$date" > /dev/null 2>&1
sudo rm /tmp/processing$date

```

create_direct.sh: Script to create the honey for the Direct File Architecture:

```
# sets up direct architecture
# takes in a container name to set up architecture
homepath=/var/lib/lxc/"$1"/rootfs
sudo mkdir $homepath/"B&B Customers"

sudo mkdir $homepath/"B&B Customers"/Names
sudo cp /home/student/fake_data/names.csv /var/lib/lxc/$1/rootfs/"B&B
Customers"/Names

sudo mkdir $homepath/"B&B Customers"/Emails
sudo cp /home/student/fake_data/emails.csv /var/lib/lxc/$1/rootfs/"B&B
Customers"/Emails

sudo mkdir $homepath/"B&B Customers"/"Account Balances"
sudo cp /home/student/fake_data/account_balances.csv
/var/lib/lxc/$1/rootfs/"B&B Customers"/"Account Balances"

sudo mkdir $homepath/"B&B Customers"/Addresses
sudo cp /home/student/fake_data/addresses.csv /var/lib/lxc/$1/rootfs/"B&B
Customers"/Addresses

sudo mkdir $homepath/"B&B Customers"/Transactions
sudo cp /home/student/fake_data/transactions.csv /var/lib/lxc/$1/rootfs/"B&B
Customers"/Transactions

sudo mkdir $homepath/"B&B Customers"/"Usernames and Passwords"
sudo cp /home/student/fake_data/usernames_passwords.csv
/var/lib/lxc/$1/rootfs/"B&B Customers"/"Usernames and Passwords"
```

create_sequential.sh: Script to create the honey for the Sequential File Architecture:

```
# sets up linear architecture
# takes in a container name to set up architecture
homepath=/var/lib/lxc/"$1"/rootfs
sudo mkdir $homepath/"B&B Customers"

sudo mkdir $homepath/"B&B Customers"/Names
sudo cp /home/student/fake_data/names.csv /var/lib/lxc/$1/rootfs/"B&B
Customers"/Names

sudo mkdir $homepath/"B&B Customers"/Names/Emails
sudo cp /home/student/fake_data/emails.csv /var/lib/lxc/$1/rootfs/"B&B
Customers"/Names/Emails

sudo mkdir $homepath/"B&B Customers"/Names/Emails/"Account Balances"
sudo cp /home/student/fake_data/account_balances.csv
/var/lib/lxc/$1/rootfs/"B&B Customers"/Names/Emails/"Account Balances"

sudo mkdir $homepath/"B&B Customers"/Names/Emails/"Account Balances"/Addresses
sudo cp /home/student/fake_data/addresses.csv /var/lib/lxc/$1/rootfs/"B&B
Customers"/Names/Emails/"Account Balances"/Addresses

sudo mkdir $homepath/"B&B Customers"/Names/Emails/"Account
Balances"/Addresses/Transactions
sudo cp /home/student/fake_data/transactions.csv /var/lib/lxc/$1/rootfs/"B&B
Customers"/Names/Emails/"Account Balances"/Addresses/Transactions

sudo mkdir $homepath/"B&B Customers"/Names/Emails/"Account
Balances"/Addresses/Transactions/"Username and Passwords"
sudo cp /home/student/fake_data/usernames_passwords.csv
/var/lib/lxc/$1/rootfs/"B&B Customers"/Names/Emails/"Account
Balances"/Addresses/Transactions/"Username and Passwords"
```


create_tree.sh: Script to create the honey for the Tree File Architecture:

```
# sets up binary architecture
# takes in a container name to set up architecture
homepath=/var/lib/lxc/"$1"/rootfs
sudo mkdir $homepath/"B&B Customers"

sudo mkdir $homepath/"B&B Customers"/"Personal Information"
sudo mkdir $homepath/"B&B Customers"/"Financial Information"

sudo mkdir $homepath/"B&B Customers"/"Personal Information"/Addresses
sudo cp /home/student/fake_data/addresses.csv /var/lib/lxc/$1/rootfs/"B&B
Customers"/"Personal Information"/Addresses

sudo mkdir $homepath/"B&B Customers"/"Personal Information"/Names
sudo cp /home/student/fake_data/names.csv /var/lib/lxc/$1/rootfs/"B&B
Customers"/"Personal Information"/Names

sudo mkdir $homepath/"B&B Customers"/"Personal Information"/Names/Emails
sudo cp /home/student/fake_data/emails.csv /var/lib/lxc/$1/rootfs/"B&B
Customers"/"Personal Information"/Names/Emails

sudo mkdir $homepath/"B&B Customers"/"Financial Information"/Transactions
sudo cp /home/student/fake_data/transactions.csv /var/lib/lxc/$1/rootfs/"B&B
Customers"/"Financial Information"/Transactions

sudo mkdir $homepath/"B&B Customers"/"Financial
Information"/Transactions/"Account Balances"
sudo cp /home/student/fake_data/account_balances.csv
/var/lib/lxc/$1/rootfs/"B&B Customers"/"Financial
Information"/Transactions/"Account Balances"

sudo mkdir $homepath/"B&B Customers"/"Financial Information"/"Usernames and
Passwords"
sudo cp /home/student/fake_data/usernames_passwords.csv
/var/lib/lxc/$1/rootfs/"B&B Customers"/"Financial Information"/"Usernames and
Passwords"
```

Works Cited

- Barron, Timothy, and Nick Nikiforakis. Picky Attackers. Annual Computer Security Applications Conference, Dec. 2017, <https://doi.org/10.1145/3134600.3134614>. Accessed 15 September 2023.
- Cardenas, Alfonso F. Evaluation and Selection of File Organization---a Model and System. Communications of the ACM, vol. 16, no. 9, Sept. 1973, pp. 540–48, <https://doi.org/10.1145/362342.362352>. Accessed 15 September 2023.
- S. Dowling, M. Schukat and H. Melvin, "A ZigBee honeypot to assess IoT cyberattack behaviour," 2017 28th Irish Signals and Systems Conference (ISSC), Killarney, Ireland, 2017, pp. 1-6, doi: 10.1109/ISSC.2017.7983603.
- Dumont, Pierre, et al. Detection of Malicious Remote Shell Sessions. International Conference on Cyber Conflict (CyCon), May 2019, <https://doi.org/10.23919/cycon.2019.8757163>. Accessed 15 September 2023.